API Backward Compatibility

breaking changes and expand-contract

Watch the 4-min video

In one line: every response must still work for every client version in the wild; make only additive changes, keep clients tolerant, never repurpose a field, and remove things with expand-contract.

1 Why it matters

Your mobile app has **five versions in the wild**, and some users never update. Every response your API sends must work for all of them: last week's app and the one from two years ago. **Backward compatibility** is the discipline that makes that possible: a new server must not break an old client.

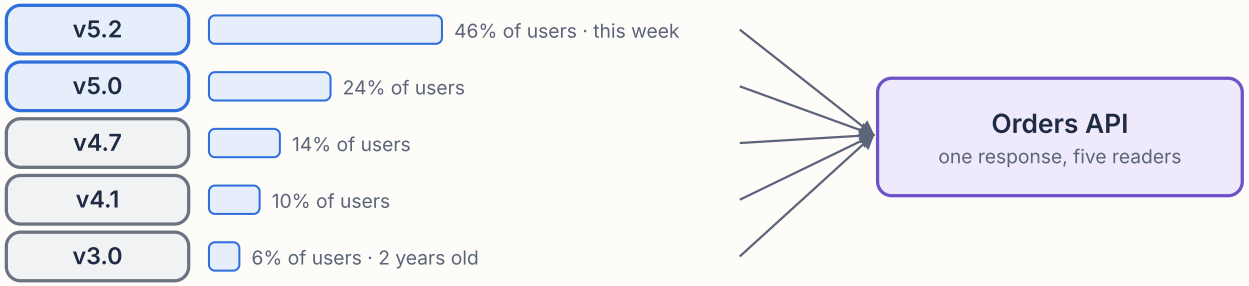


Figure 1. A typical spread of app versions. The server ships whenever it likes; the oldest client decides what it may still change.

2 What breaks, and what is safe

Breaking	Example
Removing or renaming a field	`total` → `total_amount`
Changing a type	`"id": 17` → `"id": "17"`
Changing meaning or units	`duration` in seconds → milliseconds
Making an optional input required	old clients don't send it, so they get `400`
Tightening validation	name max length 100 → 50
Adding an enum value old clients can't handle	`status: "on_hold"` crashes a strict switch

Safe: adding optional response fields, new endpoints, new optional parameters, and loosening validation (accepting more than before).

WORKED EXAMPLE

You add an optional `delivery_eta` field to the order response. Old apps simply ignore it; new apps use it. On the request side, a new optional field needs a **sensible default**, so an old app that never sends it still gets the old behavior: a new `gift_wrap` flag defaults to `false`, and the order is placed exactly as before.

3 Tolerant readers

Additive changes only work if clients are **tolerant readers**. A tolerant reader:

- **ignores unknown fields**;
- **doesn't depend on field order**;
- **handles unknown enum values gracefully**, for example as "other".

So configure JSON parsers not to fail on unknown properties, and give every `switch` on an enum a **default branch**. The old rule (Postel's law): *be conservative in what you send, and liberal in what you accept*.

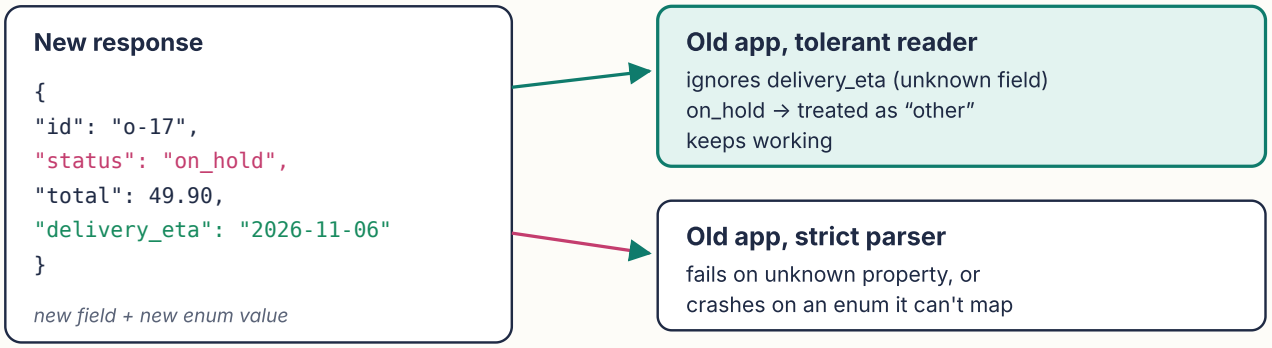


Figure 2. The same response, two old clients. The tolerant one keeps working; the strict one turns a harmless server change into a crash.

```
// Kotlin + kotlinx.serialization: don't fail on unknown fields
val json = Json { ignoreUnknownKeys = true }

// Map unknown enum values to a fallback instead of crashing
when (order.status) {
  "paid"    -> showPaid()
  "shipped" -> showShipped()
  else      -> showGeneric()   // on_hold, and anything added later
}
```

4 Never repurpose a field

If `status` used to mean **payment** status and you start using it for **shipping** status, old clients make **wrong decisions silently**, which is worse than crashing: a crash gets noticed, a wrong "paid" badge does not. **Add a new field with a new name** instead (`shipping_status`).

The same goes for units: if you need milliseconds, add `duration_ms` and keep `duration` in seconds. Putting the unit in the name makes the meaning impossible to misread.

5 Removing a field safely: expand and contract

1. **Expand.** Add the replacement field and send both.
2. **Migrate.** Move clients to the new field and mark the old one `deprecated: true` in the OpenAPI spec.
3. **Measure.** Log which clients still read or send the old field, by app version.
4. **Contract.** Only when usage is near zero *and* the minimum supported app version has moved past it, remove the field, or save the removal for the next major version.

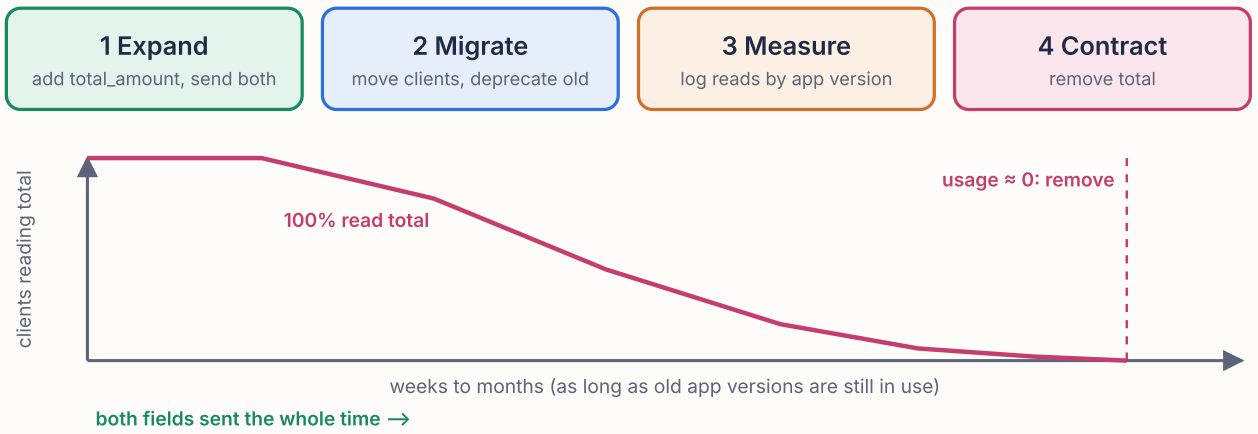


Figure 3. Expand-contract replaces one risky change with several safe ones. The removal waits for data, not for a date someone guessed.

6 Mobile: you can't force updates

- **Send the app version in a header** (for example `X-App-Version: 4.1.0`) on every request, so logs show who uses what.
- **Keep a minimum supported version**, and show a **force-update screen** for apps below it.
- **Server-side feature flags** make new behavior opt-in per app version: only 5.2+ gets the new checkout flow.
- **Keep contract tests for every version you still support**, so a change that breaks v4.1 fails CI.

COMMON MISTAKES

- Renaming a field "because the old name was ugly" in a minor release.
- Reusing a field or enum value for a new meaning.
- Strict JSON parsing in clients, so any new field crashes old apps.
- Removing a deprecated field on a date, without checking who still reads it.
- Changing units silently (seconds → milliseconds) without a new field name.

7 Key terms

Backward compatible: new server still works for old clients

Breaking change: a change that can break an existing client

Additive change: adds without removing or altering

Tolerant reader: client that ignores what it doesn't know

Expand-contract: add new, migrate, measure, then remove old

Deprecation: marking a field as going away

Minimum supported version: oldest app version still served

Feature flag: server switch to enable behavior per version

Postel's law: strict in sending, liberal in accepting

8 Interview questions

Q1. Which API changes are breaking? BASIC

Removing or renaming fields; changing types, meanings or units; making inputs required; tightening validation; and adding enum values old clients can't handle. Adding optional fields and endpoints is safe, as long as clients are tolerant readers.

Q2. How do you remove a field safely? BASIC

Expand and contract: add the replacement and send both, deprecate the old field in the spec, measure who still uses it by app version, and remove it only when usage is near zero or in a new major version.

Q3. What is a tolerant reader? BASIC

A client that ignores unknown fields, doesn't depend on field order, and maps unknown enum values to a fallback like “other”. It is what makes additive server changes safe.

Q4. You must change duration from seconds to milliseconds. How? SCENARIO

Don't change it. Add duration_ms, keep duration in seconds and send both, migrate clients, measure, and remove duration only when no client reads it. Same value, two names: 90 and 90000.

Q5. Logs show 3% of requests still come from app v3.0, which reads the old field. What do you do? SCENARIO

Keep sending the field for now. Decide whether v3.0 is below your minimum supported version; if so, show those users a force-update screen, wait until usage falls near 0%, then remove the field.

Q6. You add status = "on_hold" and old Android apps crash. What went wrong and what is the fix now? SCENARIO

The old parser had no default branch for unknown enum values. Short term, stop sending on_hold to old app versions (map it to an existing value by X-App-Version, or behind a flag). Long term, ship tolerant enum handling and contract-test old versions.

Q7. Why is repurposing a field worse than removing it? TRICKY

Removing breaks loudly: a crash or a missing value you'll see in error logs. Repurposing breaks silently: old clients read “status: paid” as payment status when it now means shipping, and make wrong decisions nobody notices.

Q8. Is loosening validation always safe? TRICKY

For old clients, yes: everything they sent before is still accepted. But check downstream: if you raise a name limit from 50 to 200 characters, the database column, other services and old clients that display the value must handle 200 too.

Next: SD-E-001 · Relational Model & SQL Basics backendroadmap.com